REACTNATIVEPLAYGROUND · LEARNING GUIDE · STEP 1 OF 2

Home tab: real categories and products from the API

Replace HomeTab's mock data with `GET /categories` and `GET /products`, using typed models that the cart guide (step 2) builds on.

October 27, 2026 · React Native 0.87 · axios 1.20 · Every file in this guide was typechecked, bundled and tested against the live API

The plan

What changes

File	Change
`src/api/models/response/listResponse.ts`	new: the wrapper every list endpoint uses
`src/api/models/response/categoryResponse.ts`	new: `CategoryDto`
`src/api/models/response/productResponse.ts`	new: `ProductDto`
`src/api/errors.ts`	new: turns an axios error into the server's message
`src/api/apiServices.ts`	add `getCategories()` and `getProducts()`
`src/screens/main_tabs/tabs/home_tab/HomeTab.tsx`	load both lists from the API, with a spinner and an error toast
`src/models/Category.ts`, `src/models/Product.ts`	**delete**: nothing uses the mocks anymore

The two endpoints

Neither needs a token. Both wrap their list the same way:

```
// GET https://ecommerce.routemisr.com/api/v1/categories
{
    "results": 10,
    "metadata": {"currentPage": 1, "numberOfPages": 1, "limit": 40},
    "data": [
        {"_id": "6439d61c0049ad0b52b90051", "name": "Music", "slug": "music",
         "image": "https://ecommerce.routemisr.com/Route-Academy-categories/1681511964020.jpeg",
         "createdAt": "2023-04-14T22:39:24.365Z", "updatedAt": "2023-04-14T22:39:24.365Z"},
        ...
    ]
}

// GET https://ecommerce.routemisr.com/api/v1/products
{
    "results": 56,
    "metadata": {"currentPage": 1, "numberOfPages": 2, "limit": 40, "nextPage": 2},
    "data": [ { "_id": "...", "title": "Woman Shawl", "price": 149, "imageCover": "...", ... }, ...
]
}
```

How this guide was checked

I applied every change to a copy of your current project. The code blocks marked with a file path are those exact files, inserted by the PDF builder, so nothing was retyped.

- **TypeScript:** no new errors. The old `Category.ts` mock-id errors are gone, and only the existing `LoginScreen.tsx:47` error remains (see the last page).
- **Metro:** a full production bundle builds.
- **Live API:** the same calls the app makes return 10 categories, and `limit: 10` returns 10 products with `title`, `price`, `ratingsAverage` and `imageCover`. With no limit you get 40 of the 56 products.

Not done: running on a simulator.

1 · The models

1.1 · The list wrapper (shared)

Every Route list endpoint returns `results`, `metadata` and `data`. Define it once as a **generic**, like a `BaseResponse<T>` class in Flutter. `ListResponse<CategoryDto>` means "the wrapper, with an array of categories inside".

> src/api/models/response/listResponse.ts

```
// Every Route list endpoint (/products, /categories, /brands) wraps its data like this
export interface Metadata {
    currentPage: number;
    numberOfPages: number;
    limit: number;
    nextPage?: number;   // only when there is a next page
}

export interface ListResponse<T> {
    results: number;
    metadata: Metadata;
    data: T[];
}
```

1.2 · Categories

> src/api/models/response/categoryResponse.ts

```
import {ListResponse} from "./listResponse.ts";

// One item of GET /categories
export interface CategoryDto {
    _id: string;
    name: string;
    slug: string;
    image: string;
    createdAt: string;   // ISO date string, e.g. "2023-04-14T22:39:24.365Z"
    updatedAt: string;
}

export type CategoriesResponse = ListResponse<CategoryDto>;
```

- `type CategoriesResponse = ListResponse<CategoryDto>` is a **type alias**, a short name for the full type. It creates nothing new at runtime.
- **Dates are strings:** JSON has no date type. Use `new Date(category.createdAt)` when you need a real date.

1.3 · Products

src/api/models/response/productResponse.ts

```ts
import {ListResponse} from "./listResponse.ts";

// Shapes from GET /products and GET /products/:id (only the fields we use).

export interface CategoryRefDto {
    _id: string;
    name: string;
    slug: string;
    image: string;
}

export interface BrandRefDto {
    _id: string;
    name: string;
    slug: string;
    image: string;
}

export interface SubcategoryRefDto {
    _id: string;
    name: string;
    slug: string;
    category: string;   // the parent category's id
}

export interface ProductDto {
    _id: string;
    id: string;                    // the API sends the id twice: `_id` and `id`
    title: string;
    slug: string;
    price: number;
    priceAfterDiscount?: number;    // only on discounted products
    quantity: number;               // stock left
    sold: number;
    imageCover: string;
    ratingsAverage: number;
    ratingsQuantity: number;
    category: CategoryRefDto;
    brand: BrandRefDto;
    subcategory: SubcategoryRefDto[];
}

export type ProductsResponse = ListResponse<ProductDto>;
```

- **Use the API's field names** (`title`, `imageCover`, `ratingsAverage`) instead of renaming them. The cart guide stores these same objects, so keeping the names saves mapping code everywhere.
- `priceAfterDiscount?` is optional because the server only sends it for discounted products. `?` means "the key may be missing". It's different from `| null`, which means "the key is always there, and its value may be null".
- **Only the fields you use.** The API also sends `images`, `description`, `createdAt` and more. Add them to the interface when you need them. An interface is only a compile-time description, so extra JSON fields don't break anything.
- **No `fromJson`:** axios has already parsed the JSON into plain objects. The interface just tells TypeScript what's in them.

> **Flutter analogy:** each `interface` is a model class without `fromJson`. `ListResponse<T>` is `class BaseListResponse<T> { List<T> data; ... }`.

2 · The API calls

2.1 · Error messages

src/api/errors.ts

```
import axios from "axios";

// Route error bodies look like {"statusMsg": "fail", "message": "..."}
// or, for validation errors, {"message": "fail", "errors": {"msg": "..."}}
interface RouteErrorBody {
    message?: string;
    errors?: { msg?: string };
}

// Turns any error into a message we can show in a Toast
export function getErrorMessage(error: unknown): string {
    if (axios.isAxiosError<RouteErrorBody>(error)) {
        const body = error.response?.data;
        return body?.errors?.msg ?? body?.message ?? error.message;
    }
    return "Something went wrong";
}
```

`axios.isAxiosError(error)` is a type guard, like `on DioException catch (e)`. When the server replied, its message is in `error.response.data`. When it didn't (offline, timeout), axios's own message is used, such as "Network Error".

2.2 · `getCategories()` and `getProducts()`

Add two imports at the top of `apiServices.ts` and two functions under `login`. The rest of your file, including your logging interceptors, stays as it is:

src/api/apiServices.ts

```
// var dio = Dio(BaseOptions(baseUrl: "", headers))

import axios, {AxiosError, AxiosResponse, InternalAxiosRequestConfig} from "axios";
import {AuthResponse, User} from "./models/response/authResponse.ts";
import {tokenStorage} from "../core/async_storage/asyncStorageHelper.ts";
import {CategoriesResponse, CategoryDto} from "./models/response/categoryResponse.ts";
import {ProductDto, ProductsResponse} from "./models/response/productResponse.ts";

const apiClient = axios.create({
    baseURL: 'https://ecommerce.routemisr.com/api/v1/',
    headers: {
        'Content-Type': 'application/json',
    }
})
```

```
export async function login(email: string, password: string): Promise<any> {
    const response = await apiClient.post<AuthResponse>("auth/signin", {email, password});
    if (response.data.token) {
        await tokenStorage.save(response.data.token);
    }
    return response.data;
}

export async function getCategories(): Promise<CategoryDto[]> {
    const response = await apiClient.get<CategoriesResponse>("categories");
    return response.data.data;   // axios body → the API's `data` array
}

// limit = how many products to load (the API's default is 40)
export async function getProducts(limit?: number): Promise<ProductDto[]> {
    const response = await apiClient.get<ProductsResponse>("products", {params: {limit}});
    return response.data.data;
}


// 1. REQUEST INTERCEPTOR
apiClient.interceptors.request.use(
    (config: InternalAxiosRequestConfig) => {
        const method = config.method?.toUpperCase() || 'METHOD';

        console.group(`🚀 [API Request] ${method} → ${config.url}`);

        // Log headers safely (avoids [object Object])
        if (config.headers) {
            console.log('Headers:', JSON.parse(JSON.stringify(config.headers)));
        }

        // Log request body/params if they exist
        if (config.data) {
            console.log('Body:', config.data);
        }
        if (config.params) {
            console.log('Params:', config.params);
        }

        console.groupEnd();
        return config;
    },
    (error: AxiosError) => {
        console.error('❌ [API Request Error]', error);
        return Promise.reject(error);
    }
```

```
);

// 2. RESPONSE INTERCEPTOR
apiClient.interceptors.response.use(
    (response: AxiosResponse) => {
        console.group(`✅ [API Response] ${response.status} → ${response.config.url}`);

        // Log the actual server payload data
        if (response.data) {
            console.log('Data:', response.data);
        }

        console.groupEnd();
        return response;
    },
    (error: AxiosError) => {
        // Gracefully handle server responses with error codes (400, 401, 500, etc.)
        if (error.response) {
            console.group(`❌ [API Error Response] ${error.response.status} →
${error.config?.url}`);
            console.log('Error Data:', error.response.data);
            console.groupEnd();
        } else if (error.request) {
            // The request was made but no response was received (e.g., Network Down)
            console.error('📡 [API Network Error] No response received:', error.request);
        } else {
            console.error('⚙️ [API Setup Error]:', error.message);
        }

        return Promise.reject(error);
    }
);
```

- `apiClient.get<CategoriesResponse>("categories")`: the generic tells TypeScript what the JSON body contains. The URL is relative to your `baseURL`, which ends in `/`, so there's no leading slash.
- **Why `response.data.data`?** The first `.data` is axios's wrapper around the body, and the second is the API's own `data` array.
- `{params: {limit}}`: axios builds `?limit=10`. If `limit` is `undefined`, axios leaves it out, and the API returns its default page of 40.
- **The functions return the array**, not the whole response, so screens get `CategoryDto[]` directly. When you need paging later, return `response.data` (the full `ListResponse`) instead and read `metadata.nextPage`.

3 · HomeTab

3.1 · The pattern: load once, then show

```
const [categories, setCategories] = useState<CategoryDto[]>([]);   // the data
const [loading, setLoading] = useState(true);                       // spinner flag

useEffect(() => {                    // runs once, after the first render (like initState)
    getCategories()
        .then(setCategories)         // success: store the list → re-render with data
        .catch(error => Toast.show({type: "error", text1: "Categories", text2:
getErrorMessage(error)}))
        .finally(() => setLoading(false));   // always: hide the spinner
}, []);                              // [] = no dependencies = run only once
```

- `.then(setCategories)` is shorthand for `.then(list => setCategories(list))`. A function can be passed directly when its argument fits.
- `.finally` runs after success *or* failure, so the spinner never gets stuck.
- **The toast** uses the `<Toast/>` you already render in `App.tsx`.

> **Flutter analogy:** `useState` + `useEffect(…, [])` is a `StatefulWidget` that starts a `Future` in `initState` and calls `setState` when it completes. It's what a `FutureBuilder` does for you.

3.2 · What changes in HomeTab.tsx

- **Imports:** the mock imports (`categories`, `Category`, `mockProducts`, `Product`) are replaced by `useEffect` / `useState`, `ActivityIndicator`, `Toast`, the two API functions, `getErrorMessage` and the two DTO types.
- **`Categories`:** loads from the API, shows a spinner while loading, and builds the pairs from the loaded list. `cateogryPairs` is also renamed to `categoryPairs`.
- **`CategoryItem`:** takes `{category}` and shows the real image (`{uri: category.image}`) instead of `ads_img.png`. A fixed width and centered text keep long names like "Men's Fashion" tidy.
- **`ProductsSection`:** loads 10 products from the API, with a spinner.
- **`ProductItem`:** `Product` → `ProductDto`, `name` → `title`, `rating` → `ratingsAverage`, and the local image → `{uri: product.imageCover}`. The **+** stays as it is until the cart guide.
- **Keys:** `keyExtractor` uses the real `_id` instead of the index, and each `CategoryItem` inside `.map()` gets `key={category._id}`. That removes React's "*Each child in a list should have a unique key*" warning.

Remote images need a size

`source={{uri: …}}` loads an image from the network. Unlike `require(...)`, React Native doesn't know a remote image's size in advance, so it must have a `width` and `height` in its style, or it renders at 0×0. Both images here already set them.

3.3 · The full file

Here's your `HomeTab.tsx` with the changes applied. Everything else in it, including the search bar, the carousel and your styles, is unchanged.

src/screens/main_tabs/tabs/home_tab/HomeTab.tsx

```
import {useEffect, useState} from "react";
import {ActivityIndicator, FlatList, Image, ScrollView, StyleSheet, Text, TextInput,
useWindowDimensions, View} from "react-native";
import Toast from "react-native-toast-message";
import {getCategories, getProducts} from "../../../../api/apiServices.ts";
import {getErrorMessage} from "../../../../api/errors.ts";
import {CategoryDto} from "../../../../api/models/response/categoryResponse.ts";
import {ProductDto} from "../../../../api/models/response/productResponse.ts";
import {Carousel} from "react-native-reanimated-carousel";
import {SafeAreaView} from "react-native-safe-area-context";

// Text -> Text
// Button - Button
// Images - Image
// Column - Row - Stack -> View
// ListView.builder -> Flatlist
// SingleChildScrollView -> ScrollView
//TextField -> TextInput
export const blue = '#004182'

function HomeTab() {
    return (
        <SafeAreaView edges={['top']}>
            <ScrollView>
                <View style={styles.screenContainer}>
                    <Image source={require("../../../../assets/images/route_logo.png")}
                           style={styles.logo}/>
                    <SearchBarAndCartRow/>
                    <AdsCarosuel/>
                    <Categories/>
                    <ProductsSection/>
                </View>
            </ScrollView>
        </SafeAreaView>

      )
}

function SearchBarAndCartRow() {
    return (<View style={{flexDirection: 'row', alignItems: 'center', gap: 8}}>
        <View style={styles.searchContainer}>
            <Image source={require("../../../../assets/images/ic_search.png")}/>

            <TextInput placeholder='What do u search for?'
                       placeholderTextColor={blue} style={{color: blue}}/>
        </View>
        <Image source={require("../../../../assets/images/ic_cart.png")}/>
```

```
    </View>)
}

const ads = [
    require("../../../../assets/images/ads_img.png"),
    require("../../../../assets/images/ads_img.png"),
    require("../../../../assets/images/ads_img.png"),
]

function AdsCarosuel() {
    const {width} = useWindowDimensions();
    const fullWidth = width - 32
    return (<Carousel
        style={{height: 200, width: fullWidth}}
        data={ads}
        autoplay={true}
        autoplayInterval={2000}
        renderItem={({item}) => (
            <View style={{alignItems: "center", justifyContent: "center", }}>
                <Image style={{marginHorizontal: 16, padding:16, width: fullWidth - 16}} source=
{item}/>
            </View>
        )}
    />)
}

function Categories() {
    const [categories, setCategories] = useState<CategoryDto[]>([]);
    const [loading, setLoading] = useState(true);

    // Runs once when the section appears, like initState
    useEffect(() => {
        getCategories()
            .then(setCategories)
            .catch(error => Toast.show({type: "error", text1: "Categories", text2:
getErrorMessage(error)}))
            .finally(() => setLoading(false));
    }, []);

    // [a, b, c, d, e] → [[a, b], [c, d], [e]] for the two-row horizontal list
    const categoryPairs: CategoryDto[][] = [];
    for (let i = 0; i < categories.length; i += 2) {
        categoryPairs.push(categories.slice(i, i + 2))
    }

    return (<View>
        <View style={{flexDirection: 'row', justifyContent: 'space-between', gap: 8}}>
            <Text style={{color: blue}}>Categories</Text>
            <Text style={{color: blue}}>View all</Text>
```

```
        </View>
        {loading ? <ActivityIndicator color={blue} style={{marginTop: 16}}/> : (
            <FlatList
                horizontal={true}
                data={categoryPairs}
                showsHorizontalScrollIndicator={false}
                renderItem={({item}) => <View style={{gap: 8}}>
                    {item.map((category) => <CategoryItem key={category._id} category={category}/>)}
                </View>}
                keyExtractor={(pair) => pair[0]._id}
            />
        )}
    </View>)
}

function CategoryItem({category}: { category: CategoryDto }) {
    return (<View style={{marginHorizontal: 16, width: 100, alignItems: 'center'}}>
        <Image source={{uri: category.image}} style={{width: 100, height: 100, borderRadius: 50}}/>
        <Text style={{color: blue, textAlign: 'center'}} numberOfLines={2}>{category.name}</Text>
    </View>)
}

function ProductsSection() {
    const [products, setProducts] = useState<ProductDto[]>([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        getProducts(10)
            .then(setProducts)
            .catch(error => Toast.show({type: "error", text1: "Products", text2:
getErrorMessage(error)}))
            .finally(() => setLoading(false));
    }, []);

    return (<View>
        <View style={{flexDirection: 'row', justifyContent: 'space-between', gap: 8}}>
            <Text style={{color: blue}}>Products</Text>
            <Text style={{color: blue}}>View all</Text>
        </View>
        {loading ? <ActivityIndicator color={blue} style={{marginTop: 16}}/> : (
            <FlatList
                horizontal={true}
                data={products}
                showsHorizontalScrollIndicator={false}
                renderItem={({item}) => <ProductItem product={item}/>}
                keyExtractor={(item) => item._id}
            />
        )}
    </View>)
```

```
}

function ProductItem({product}: { product: ProductDto }) {
    return (<View style={{
        borderRadius: 16,
        // borderColor: '#0041824D',
        width: 190,
        marginHorizontal: 8,
        marginTop: 16,
        borderWidth: 1,
        paddingBottom: 8
    }}>
        <Image source={{uri: product.imageCover}}
               style={{width: '100%', height: 120, borderTopLeftRadius: 16, borderTopRightRadius:
16}}/>
        <View style={{marginHorizontal: 8, gap: 8}}>
            <Text style={{color: blue,}} numberOfLines={2}>{product.title}</Text>
            <Text style={{color: blue}}>EGP {product.price}</Text>
            <View style={{flexDirection: 'row', justifyContent: 'space-between', alignContent:
'center',}}>
                <Text style={{color: blue, verticalAlign: "middle"}}>Review
({product.ratingsAverage})</Text>
                <Text style={{
                    color: 'white',
                    backgroundColor: blue,
                    width: 30,
                    height: 30,
                    borderRadius: 15,
                    textAlign: 'center',
                    verticalAlign: "middle",
                    fontSize: 18
                }}>+</Text>
            </View>
        </View>

        <Image style={{position: 'absolute', right: 0}} source=
{require("../../../../assets/images/ic_favorite.png")}/>

    </View>)
}

const styles = StyleSheet.create({
    screenContainer: {
        marginHorizontal: 16,
        flex: 1,
        gap: 18
    },
    searchContainer: {
        flexDirection: 'row',
```

```
        borderColor: blue,
        borderWidth: 1,
        borderRadius: 24,
        paddingHorizontal: 18,
        paddingVertical: 8,
        alignItems: 'center',
        gap: 4,
        flex: 1
    },
    logo: {
        marginTop: 12
    }
})
export default HomeTab;
```

3.4 · Delete the mock models

Nothing imports `src/models/Category.ts` or `src/models/Product.ts` anymore, so delete both files. That also removes the old `Type 'number' is not assignable to type 'string'` errors in `Category.ts`. In WebStorm, **Find Usages** on a file confirms nothing uses it before you delete it.

> **Checklist**
> ☐ `listResponse.ts` ☐ `categoryResponse.ts` ☐ `productResponse.ts` ☐ `errors.ts` ☐ `getCategories` + `getProducts` in `apiServices.ts` ☐ HomeTab changes ☐ delete `src/models/`
> **Try it:** open Home. A spinner shows in each section, then 10 categories with real images appear in two rows, and 10 products. Your request interceptor logs `GET → categories` and `GET → products`. Turn on airplane mode and reload: each section shows a "Network Error" toast.

4 · Reference

Troubleshooting

Symptom	Cause → fix
Images are blank, but the text shows	A remote image with no size. Give it `width` and `height` (3.2).
A 404 "Can't find this route"	The URL is wrong. With `baseURL` ending in `/`, use `"categories"`, not `"/categories"`.
`Cannot read property 'map' of undefined`	Something used `response.data` (the wrapper) as the list. The list is `response.data.data`.
"Each child in a list should have a unique key"	A `.map()` without `key`. Use `key={item._id}`.
Spinner never stops	`setLoading(false)` isn't in `.finally`, or the promise chain lost a `.catch`.

Next: the cart guide

The cart guide (step 2) builds on these exact models: the cart stores `ProductDto`s, and its products already come from `getProducts()`. Do this guide first.

> **An existing error in your project:** `tsc` reports `Property 'replace' does not exist` at `LoginScreen.tsx:47`. Line 22 uses the screen **props** type where the **navigation** type is needed: `useNavigation<NativeStackScreenProps<RootStackParams>>()` → `useNavigation<NativeStackNavigationProp<RootStackParams>>()`, imported from `@react-navigation/native-stack`.